quences. For two given dummy sequences $\mathbf{a}_{1:n} = \langle a_1, \ldots, a_n \rangle$ and $\mathbf{z}_{1:m} = \langle z_1, \ldots, z_m \rangle$, $\mathbf{a}_{1:n} \oplus \mathbf{z}_{1:m}$ is defined as: $\mathbf{a}_{1:n} \oplus \mathbf{z}_{1:m} = \langle a_1, \ldots a_n, z_1, \ldots z_m \rangle$. We denote sequences of harmful responses and jailbreak inputs by $\mathbf{r}_{1:k}$ and $\tilde{\mathbf{u}}_{1:m}$, respectively. Since LLMs are specifically trained to predict the probability of the next word, we define the jailbreak goal (i.e., the objective function to be maximized) as:

$$P(\mathbf{r}_{1:k} | \tilde{\mathbf{u}}_{1:m}) = \prod_{j=1}^{k} P(r_j | \tilde{\mathbf{u}}_{1:m}, \mathbf{r}_{1:j-1}) \quad (1)$$

and the goal of defense as:

$$P(\mathbf{s}_{1:n} | \tilde{\mathbf{u}}_{1:m} \oplus \mathbf{d}_{1:l}) = \prod_{i=1}^{n} P(s_i | \tilde{\mathbf{u}}_{1:m} \oplus \mathbf{d}_{1:l}, \mathbf{s}_{1:i-1}) \quad (2)$$

where $\mathbf{s}_{1:n}$ is the refusal response to the jailbreak inputs. Finally, we assess utility degradation by:

$$P(\mathbf{h}_{1:q} | \mathbf{b}_{1:p} \oplus \mathbf{d}_{1:l}) = \prod_{k=1}^{q} P(h_k | \mathbf{b}_{1:p} \oplus \mathbf{d}_{1:l}, \mathbf{h}_{1:k-1}) \quad (3)$$

where $\mathbf{h}_{1:q}$ is the normal response for each benign queries $\mathbf{b}_{1:p}$.

The overall DPP algorithm's efficacy is evaluated by its performance in both defense against malicious queries and impact on the benign queries.

3.2 Score Evaluation

In our work, the DPP must fulfill two crucial objectives: (I) **Maximization of Refusal Score** on malicious queries and (II) **Maximization of Helpful Score** on benign queries.

To achieve (I), we use the log-likelihood of Eq. 2 and define the refusal score as follows:

$$\mathcal{S}_{D_i} = \log P(\mathbf{s}_{1:n} | \tilde{\mathbf{u}}_{1:m} \oplus \mathbf{d}_{1:l}) \quad (4)$$

where S_{D_i} denotes the refusal score attributed to the i-th DPP within the population of DPPs. The vector $\mathbf{s}_{1:n}$ represents the refusal response, $\tilde{\mathbf{u}}_{1:m}$ represents the jailbreak query, and $\mathbf{d}_{1:l}$ is our DPP.

Similarly, for (II), the inputs include benign queries combined with the same DPP as used in the refusal score calculation. Applying the log-likelihood of Eq. 3. The helpful score is formulated as:

$$\mathcal{S}_{H_i} = \log P\left(\mathbf{h}_{1:q} | \mathbf{b}_{1:p} \oplus \mathbf{d}_{1:l}\right) \quad (5)$$

where S_{H_i} represents the helpfulness score assigned to the i-th DPP within the population of DPPs. The vector $\mathbf{h}_{1:q}$ denotes the standard response, whereas $\mathbf{b}_{1:p}$ refers to the benign query. The overall score function for training DPP combines the refusal and helpful scores. These scores are weighted by the coefficients α and β, respectively, to balance their contributions within the training process:

$$\mathcal{S}_{T_i} = \alpha \cdot \mathcal{S}_{D_i} + \beta \cdot \mathcal{S}_{H_i} \quad (6)$$

3.3 DPP Training Algorithm

Using the total score defined in Sec. 3.2, we use a Hierarchical Genetic Algorithm (HGA) to optimize DPP, drawing inspiration from the AutoDAN jailbreak attack in (Liu et al., 2023). We adapt and extend HGA to iteratively refine DPP based on our defined scores, as shown in Figure 1 (b) and (c) to develop our methodology, which we call the **Defensive Prompt Patch Algorithm** (DPP Algorithm). We discuss the key difference between our DPP and AutoDAN in Appendix B.4.

Initially, we establish a baseline DPP, designated as the prototype. Without loss of generality, this prototype may take the form of either a Prefix DPP or a Suffix DPP. The relative effectiveness of each configuration is assessed in Appendix C. Following this, the prototype is subjected to K iterations of rewriting via an LLM to potentially refine the DPP, creating a population of DPP candidates. Each candidate within the population is evaluated by sampling refusal data pairs and helpful data pairs from adversarial/utility datasets to compute the total score, as formulated in Eq. 6. Details on adversarial/utility datasets in our implementation can be found in Sec. 4.1.

The DPP optimization process is conducted over I iterations for each candidate, during which the DPP algorithm executes two pivotal operations: **Sentence-Level Word Substitution** and **Paragraph-Level Sentence Swap and Mutations**.

In **Sentence-Level Word Substitution**, each sentence within the population is assigned a score calculated using Eq. 6. A certain percentage of defense prompts are retained based on their scores for further optimization. For these sentences, words are initially assigned the same score as their corresponding sentences. These scores are later adjusted based on the frequency of occurrence of each word. Words whose scores surpass a specified threshold are then randomly replaced with synonyms.

In **Paragraph-Level Sentence Swap and Mutations**, we specify a swap probability p_{swap} and

a mutation probability p_{mutate}. The defensive prompt patch, modified in the previous step, is reassessed for total score at the sentence level. Employing a methodology similar to that of sentence-level optimization, the algorithm selects parent sentences based on their scores, segments and swaps these sentences, and then conducts mutations by revising sentences using an LLM.

These processes—**Sentence-Level Word Substitution** and **Paragraph-Level Sentence Swap and Mutations**—aim to increase the diversity within the prompt patch population and enhance the likelihood of identifying the optimal patch.

The full algorithm is delineated in Algorithm 1. Ultimately, the algorithm produces an updated set of optimized DPPs, comprising K enhanced patches, and identifies the Best Defensive Prompt Patch based on the highest total score. A detailed explanation of Algorithm 1 is in Appendix D.

Algorithm 1 Defensive Prompt Patch (DPP) Algorithm

1 **Arguments:** Defensive Prompt Patch Prototype O , refusal pair (x^r, y^r), helpful pair (x^h, y^h), α and β, target LLM
2 **Initialization:** Number of optimization iteration I, batch size, $p_{crossover}$, p_{mutate}, Sentence-level iterations, Paragraph-level iterations, n umber of steps, number of parent set size
3 DPP_Set ← DPP SET GENERATION(O, K) by Alg. 2
4 **while** I is not reached **do**
5 **for** $iteration$ in sentence-level iterations **do**
6 Evaluate refusal/helpful score of each DPP with $(x^r, y^r)/(x^h, y^h)$ and target LLM
7 Final Score ← calculate the score using Eq. (6)
8 Select elite and parent prompts from DPP_Set according to Final Score
9 WordDict ← Calculate each word score using selected parent prompts by Alg. 3
10 Find synonyms for each word
11 **if** random value < WordDict[$synonym$] / sum($word\ scores$) **then**
12 Replace word with synonym
13 **end if**
14 **end for**
15 **for** $iteration$ in paragraph-level iterations **do**
16 Repeat line 6 to 8
17 Conduct crossover and mutation on selected parent prompts using Alg. 4
18 **end for**
19 New_DPP_Set ← DPP_Set ∪ New_DPP
20 Best_DPP ← Best score within New_DPP_Set
21 **end while**
22 **return** (New_DPP_Set, Best_DPP)

Best DPP selection. Algorithm 1 identifies the optimal DPP for a given pair of refusal and helpful data. Our primary objective is to find a DPP that generalizes well across different user queries. To enhance the universality of DPP, we incorporate N pairs of refusal and helpful data, sampled from their respective datasets. In each iteration of the DPP algorithm, as described earlier, a set of candidate DPPs is generated along with the best DPP for the specific data pair. This set of candidate DPPs is then used for the next pair of refusal and helpful data. By iteratively optimizing this set of DPP candidates, we aim to identify the most generalizable DPP with the best defensive and utility performance. The overall optimization procedure is detailed in Algorithm 5. Appendix C included implementation details and hyperparameter settings.

4 Experiments

We demonstrate the performance of our DPP through two perspectives: **Robustness** to standard (non-adaptive) and adaptive jailbreak attacks, **Generalization** to unforeseen jailbreak queries and different LLMs, and **Clarity** of the best-found DPPs. All final DPPs are listed in Appendix E.

4.1 Experimental Setup

Adversarial Dataset: We use the AdvBench (Zou et al., 2023), specifically the **harmful behavior instructions** [2], as jailbreak questions. Each of them is fed into a well-aligned LM (Llama-2-7B-Chat (Touvron et al., 2023)) to generate the denial responses. In our experiment, we sampled 100 jailbreak questions and recorded them with their refusal responses to create the **Adversarial Dataset**.

Utility Dataset: We use the Alpaca dataset[3] as our benchmark. For consistency with the Adversarial Dataset, we also sampled only 100 benign questions and their corresponding answers.

Language Models: We perform our jailbreak experiments on two specific LLMs: **Llama-2-7B-Chat** (Touvron et al., 2023) and **Mistral-7B-Instruct-v0.2** (Jiang et al., 2023). Llama-2-7B-Chat model is an adapted version of Llama-2-7B, specifically configured for chat-based interactions. Mistral-7B-Instruct-v0.2 model is a fine-tuned chat version of Mistral-7B-v0.2. This model demonstrates a stronger ability in performance, outperforming Llama-2-13B model on all benchmarks while maintaining proficiency in language tasks.

Jailbreak Attack Methods: We use several existing jailbreak attack methods to generate advanced malicious prompts. Specifically, for each malicious behavior statement, we apply several different types of jailbreaking attacks: (i) **Uninterpretable Jailbreak Attacks** – we used GCG (Zou et al., 2023) and Base64 (Wei et al., 2023a) to generate adversarial prompts. Specifically, GCG is

[2] https://github.com/llm-attacks/llm-attacks/blob/main/data/advbench/harmful_behaviors.csv

[3] https://github.com/gururise/AlpacaDataCleaned/blob/main/alpaca_data_cleaned_archive.json

used to generate an adversarial suffix for each malicious query. Base64 encodes each harmful query in Base64 format. (ii) **Interpretable Jailbreak Attacks** – AutoDAN (Liu et al., 2023), PAIR (Chao et al., 2023), TAP (Mehrotra et al., 2023), and ICA (Wei et al., 2023b) are interpretable attacks that we used to translate the original malicious query into a new improved malicious query. Please refer to Appendix F for more details on generating new malicious queries. (iii) **Generation-based Jailbreak Attacks** – we follow Catastrophic Attack (Huang et al., 2023) to vary the hyperparameters of the LLM to generate malicious responses for each harmful question. In our evaluation, similar to the Adversarial Dataset, we utilize 100 harmful behavior questions from AdvBench to generate new malicious queries[4], all of which will be employed in our experiments.

Jailbreak Defense Methods: We compare our DPP to Self-Reminder (Xie et al., 2023) and Goal Prioritization (Zhang et al., 2023b). They are prompt-based defenses that add defense prompts as a prefix or suffix. For the Llama-2-7B chat model, we also include another defensive suffix approach called RPO (Zhou et al., 2024). For Mistral-7B-Instruct-v0.2, instead of using RPO as a baseline, we compare the results with Plain (Default) System Prompt (Zheng et al., 2024a). We defer the discussion of our choices of baselines for the two LLMs to Appendix G. Additionally, the prompts for each defense baselines can be found in Appendix H.

Evaluation Metrics: We use the Attack Success Rate (ASR) as our primary metric for evaluating the effectiveness of jailbreak defenses. The ASR measures the proportion of malicious queries that successfully bypass the LLMs alignment and generate harmful responses. Details on how we calculate ASR can be found in Appendix I. In addition to ASR, we also use AlpacaEval (Li et al., 2023) to evaluate the utility degradation of the LLM model when defenses are employed. Specifically, we utilize the metric called Win-Rate. This involves comparing the frequency with which outputs from LLM are favored over those from a reference model, given a specific user instruction. Utilizing simulated Win-Rate offers a straightforward, comparable metric across various LLMs using the same reference model. In Appendix J, we discuss the setups of evaluating with Win-Rate.

[4]For PAIR and TAP adaptive attacks, we directly utilize the dataset provided in their code-base, which they sample 50 harmful behaviors from AdvBench.

4.2 Robustness against Non-adaptive and Adaptive Attacks

Our analysis begins with a comparative evaluation of our DPP Suffix method against established defense baselines under six distinct jailbreak attacks on the Llama-2-7B-Chat model. We delineate our findings for both non-adaptive and adaptive jailbreak attacks, reporting on Attack Success Rate (ASR), Average ASR, and Win-Rate to underscore minimal utility degradation under our method.

Non-adaptive Attacks: We generate malicious queries using the aforementioned jailbreak attacks directly from the original LLMs (i.e., without any defense). From Table 2 we can summarize the following observations. First, our method outperforms RPO with respect to ICA, AutoDAN, and GCG attacks. Specifically, it outperforms the ASR of RPO by 42% for ICA attack, 18% for AutoDAN, and 15% for GCG attack. For the Base64 attack, our method is comparable to RPO with only 1% less than RPO. Second, although Goal Prioritization is a strong defense mechanism against Base64 and GCG, it fails to defend against the AutoDAN attack, where our method is 42% better than Goal Prioritization in terms of ASR. Self-Reminder has the same performance as our method against the GCG attack and a slightly weaker performance against the Base64 attack. While our method has 10% worse defense performance under AutoDAN setting, it outperforms Self-Reminder on ICA attack by 29%. The last column of Table 2 shows the utility degradation of each defense. Our method has the best Win-Rate, 82.98%, outrunning all the other baselines. Notably, the Goal Prioritization has the lowest Win-Rate, suggesting that its defense performance comes with a high cost in utility drop. Overall, our DPP not only achieves the lowest Average ASR of 3.80% but also ensures minimal utility impact, reinforcing its standing as the most robust method among those evaluated.

Adaptive Attacks: Adaptive attack (Tramer et al., 2020) is a critical evaluation procedure for assessing defense effectiveness when the defense mechanism is known to the attack. In this study, we assume that the attacker can query the protected large language model (LLM) while defense mechanisms are active during jailbreak attempts. By "adaptive", we refer to the attacker's ability to target an LLM equipped with a DPP without prior knowledge of the specific DPP being utilized (i.e., DPP is part of the post-query system prompt used by a closed-sourced LLM service provider to im-